Programming Fundamentals

Career Paths & Computer Architecture

Seyed Ahmad Mousavi Zadeh

From source code to silicon: how programs are built and run

A visual, example-driven course

Learning outcomes

By the end of this session, learners will be able to:

- Explain what programming is and how software is built.
- Identify major career paths and their market outlook.
- Distinguish high-level and low-level languages.
- Describe how source code becomes an executable program.
- Explain the role of compilers, assemblers, linkers, and loaders.
- Differentiate hardware, software, and firmware.
- Compare RAM and ROM, primary and secondary memory.
- Describe the main components of a CPU and the fetch–decode–execute cycle.

Scope

This session builds a mental model. It is conceptual and practical, not a full course in compiler theory or digital electronics.

What is programming?

Programming is the process of designing and writing instructions that a computer can execute to solve a problem.

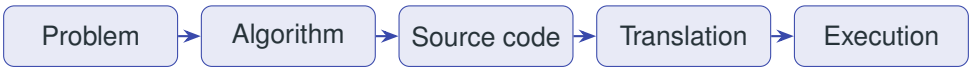
- **Problem solving:** break a task into steps.
- **Algorithm:** a precise, finite sequence of operations.
- **Data:** values the program reads, transforms, and stores.
- **Control flow:** decisions, loops, and functions.
- **Abstraction:** hide complexity behind clear interfaces.

Think → Code

An algorithm is the idea. Code is one way to express it.

Write · Test · Improve

From idea to running program



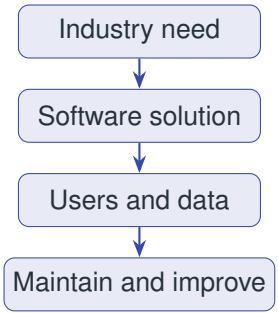
Key insight

Computers do not understand human languages. A program must be translated into machine instructions before it can run.

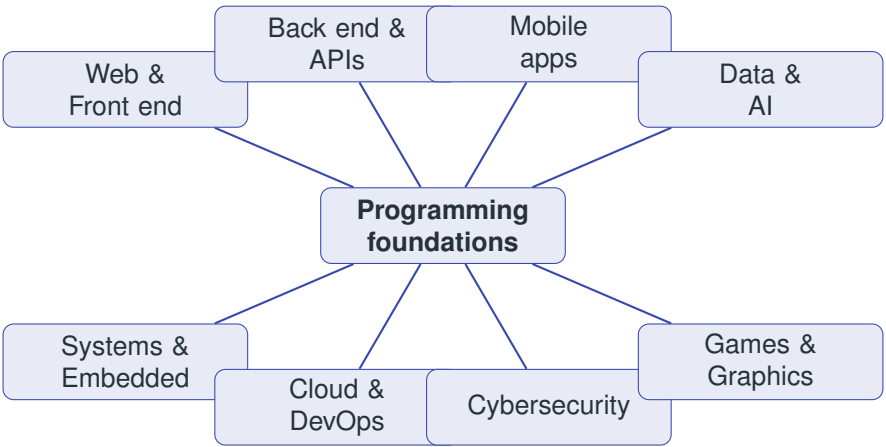
Why is programming in demand?

- Nearly every industry depends on software: health, finance, education, logistics, media.
- Digital transformation creates demand for web, mobile, data, cloud, and embedded systems.
- Automation and AI increase the need for people who can build and maintain systems.
- Skills support employment, freelancing, startups, and research.

Demand varies by region, specialization, and experience level. Portfolios, projects, communication, and continuous learning matter alongside technical skills.



Career paths: a map



Most paths share the same foundations: logic, data structures, algorithms, version control, debugging, and communication.

Career path: web and mobile development

Front end

- HTML, CSS, JavaScript, TypeScript
- Frameworks: React, Vue, Svelte
- Focus: user interface, accessibility, performance

Back end

- Node.js, Python, Java, C#, Go, PHP
- Databases, APIs, authentication
- Focus: business logic, data, security

Mobile

- Native: Swift (iOS), Kotlin (Android)
- Cross-platform: Flutter, React Native
- Focus: app UX, device APIs, app stores

Market outlook

High demand, strong freelance market, and many junior-to-senior growth paths. Portfolio projects matter heavily.

Career path: data, AI, cloud, and DevOps

Data & AI

- Python, SQL, statistics, machine learning
- Roles: data analyst, data engineer, ML engineer
- Strong growth, often requires math foundations

Cloud & DevOps

- Linux, networking, Docker, Kubernetes, CI/CD
- AWS, Azure, Google Cloud
- Focus: reliability, automation, deployment

Cybersecurity

- Networking, operating systems, scripting
- Offensive and defensive roles
- Growing demand and certification paths

Systems & embedded

- C, C++, Rust, assembly
- Microcontrollers, firmware, real-time systems
- Smaller but specialized and durable demand

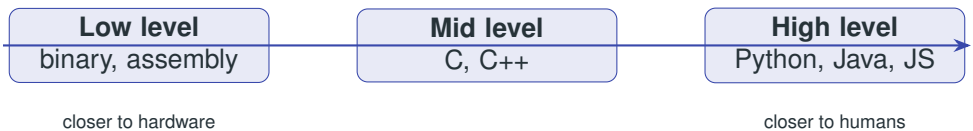
How to choose a path

1. Start with fundamentals: logic, data structures, algorithms.
2. Try small projects in two or three areas.
3. Follow curiosity, but also check local job demand.
4. Build a public portfolio and learn version control.
5. Go deep in one area after you understand the basics.

Reality check

Technology changes quickly. The durable skills are problem solving, reading code, debugging, and learning how to learn.

Programming languages: a spectrum



- **Low-level:** direct control of hardware, fast, harder to write and port.
- **High-level:** readable, productive, portable, more abstraction.
- The boundary is not strict; many languages mix features.

Low-level languages

```
; simple assembly example (x86-like)
mov eax, 5
add eax, 3
; result in eax = 8
```

- Machine code: raw binary instructions.
- Assembly: human-readable mnemonics for machine instructions.
- Used in firmware, drivers, bootloaders, embedded systems.
- Strength: speed and direct hardware access.
- Weakness: verbose, platform-specific, error-prone.

Assembly is translated to machine code by an **assembler**.

High-level languages

```python
# Python
name = "Ada"
for i in range(3):
    print("Hello,", name)
```

```java
// Java
for (int i = 0; i < 3; i++) {
    System.out.println("Hello, Ada");
}
```

- Readable and productive.
- Portable across platforms.
- Manage memory automatically in many cases.
- Examples: Python, Java, C#, JavaScript, Go, Rust.
- Trade-off: less direct hardware control, sometimes more overhead.

Compiled vs interpreted

Compiled

- Source → machine code before running
- Examples: C, C++, Rust, Go
- Fast execution, platform-specific binaries

Interpreted

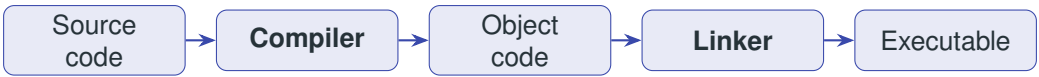
- Source executed line by line by an interpreter
- Examples: Python, Ruby, JavaScript
- Flexible, portable, often slower

Hybrid models

Java and C# compile to intermediate bytecode, then run on a virtual machine with just-in-time (JIT) compilation.

What is a compiler?

A compiler translates source code written in one language into another form, usually machine code or an intermediate representation.



- Detects syntax and semantic errors.
- Optimizes code for speed or size.
- Produces object files that the linker combines into a program.

The compilation pipeline

1. **Lexical analysis:** break source into tokens.
2. **Syntax analysis:** build a parse tree.
3. **Semantic analysis:** check types and meaning.
4. **Intermediate code:** platform-independent representation.
5. **Optimization:** improve performance or size.
6. **Code generation:** emit assembly or machine code.
7. **Assembly and linking:** produce the final executable.

Modern compilers such as GCC, Clang, and LLVM perform many optimization passes.

Compiler, interpreter, assembler, linker

Translators

- **Compiler:** whole program → target code.
- **Interpreter:** executes source directly.
- **Assembler:** assembly → machine code.

Build tools

- **Linker:** combines object files and libraries.
- **Loader:** loads the executable into memory.
- **Debugger:** inspects a running program.

Mental model

Compile → assemble → link → load → run.

Hardware vs software

Hardware

- Physical components
- CPU, memory, motherboard, storage
- Cannot change behavior without redesign

Software

- Instructions and data
- Operating systems, applications, drivers
- Can be updated and replaced

They need each other

Hardware without software is inert. Software without hardware cannot run.

Firmware: software embedded in hardware

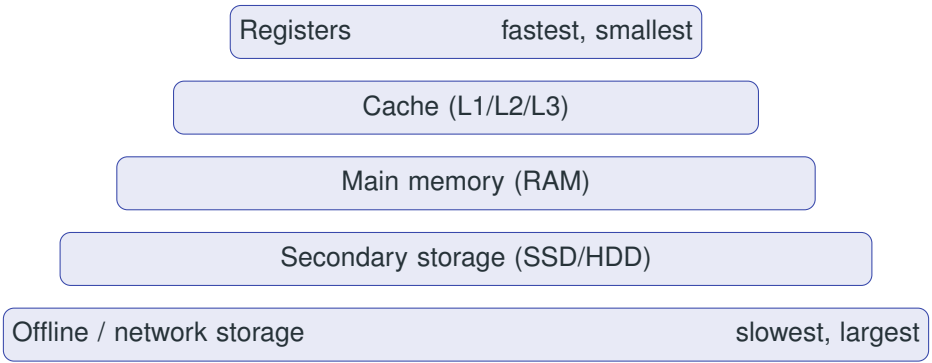
Firmware is software stored in non-volatile memory that controls hardware behavior at a low level.

- Examples: BIOS/UEFI, router firmware, printer firmware, microcontroller code.
- Usually stored in ROM, flash, or EEPROM.
- Bridges hardware and higher-level software.
- Updates can fix bugs and add features, but a failed update can brick a device.

Summary

Hardware is the body, software is the mind, and firmware is the reflex system.

Memory hierarchy



Faster memory is more expensive per byte, so computers use small fast memory near the CPU and large slow memory farther away.

Primary memory

Primary memory is directly accessible by the CPU and holds data currently in use.

RAM

- Random Access Memory
- Volatile: loses data when power is off
- Holds running programs and active data

ROM

- Read-Only Memory
- Non-volatile: keeps data without power
- Stores firmware and startup code

Cache memory is also primary but is not usually listed as RAM or ROM; it sits between the CPU and RAM.

RAM vs ROM

Property	RAM	ROM
Full name	Random Access Memory	Read-Only Memory
Volatility	Volatile	Non-volatile
Typical use	Running programs	Firmware, boot code
Writable?	Yes	Rarely (flash/EEPROM)
Speed	Fast	Slower than RAM

Modern ROM is often flash memory that can be updated, but the name persists.

Secondary memory

Secondary storage keeps data persistently, even without power.

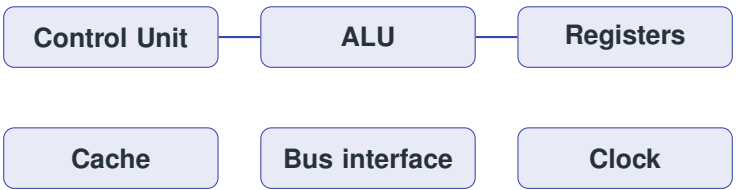
- **HDD:** magnetic disks, large capacity, lower cost per byte.
- **SSD:** flash memory, much faster, no moving parts.
- **USB flash drives and memory cards:** portable storage.
- **Optical and tape:** archival or specialized use.

Trade-off

Secondary storage is slower than RAM but much larger and cheaper per gigabyte. The operating system moves data between RAM and storage as needed.

CPU: the brain of the computer

The Central Processing Unit executes instructions and coordinates other components.



Modern CPUs may have multiple cores, each with its own registers and cache, sharing main memory.

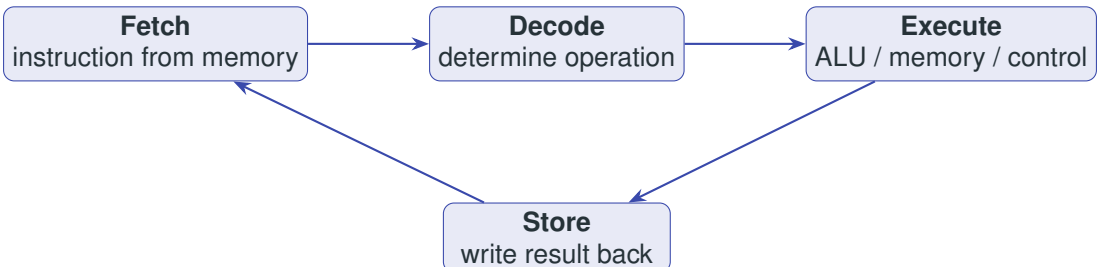
CPU components

- **Control Unit (CU):** fetches, decodes, and directs execution.
- **Arithmetic Logic Unit (ALU):** performs arithmetic and logic operations.
- **Registers:** tiny, very fast storage inside the CPU.
- **Cache:** small fast memory close to the CPU.
- **Clock:** drives the timing of operations (measured in GHz).
- **Bus interface:** connects the CPU to memory and devices.

Registers you may hear about

Program counter (PC), instruction register (IR), accumulator, stack pointer, general-purpose registers.

The fetch–decode–execute cycle



The program counter points to the next instruction. Pipelining allows several instructions to be processed at different stages simultaneously.

A tiny instruction example

```
// high-level
int a = 5;
int b = 3;
int c = a + b;
```

```
; conceptual assembly
load  R1, a
load  R2, b
add   R3, R1, R2
store c, R3
```

What the CPU does

It loads values into registers, performs the addition in the ALU, and stores the result back to memory.

Pipelining, cores, and performance

Pipelining

- Overlap instruction stages
- Improves throughput
- Hazards require stalls or forwarding

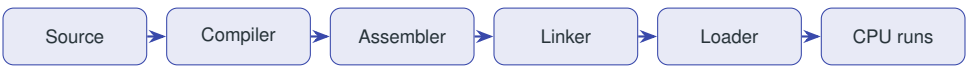
Multiple cores

- Several CPUs on one chip
- True parallel execution
- Software must be designed for concurrency

Performance factors

Clock speed, instructions per cycle, cache size, number of cores, memory bandwidth, and software efficiency all matter.

Full journey: source code to execution



- The loader places the program in memory and sets the program counter.
- The CPU then fetches and executes instructions one by one.

The operating system’s role

The operating system manages hardware and provides services to programs.

- Process and memory management.
- File systems and device drivers.
- Scheduling CPU time between programs.
- Networking and security boundaries.
- Providing system calls and APIs.

Why it matters

Most programs do not talk to hardware directly. They ask the operating system, which enforces safety and sharing.

Binary: how computers represent data

Computers store and process data as binary digits (bits): 0 and 1.

- 8 bits = 1 byte.
- Text is encoded with standards such as ASCII or Unicode (UTF-8).
- Numbers use binary; negative numbers often use two's complement.
- Images, audio, and video are encoded as structured binary data.



one byte = 8 bits

Algorithms and complexity

An algorithm is a finite sequence of well-defined steps that solves a problem.

- **Correctness:** does it produce the right result?
- **Efficiency:** how do time and memory grow with input size?
- **Big-O notation:** describes upper-bound growth, e.g., $O(n)$, $O(n^2)$, $O(\log n)$.
- Choosing the right data structure often matters more than micro-optimizations.

Example

Searching a sorted list with binary search is $O(\log n)$; linear search is $O(n)$.

Core data structures

Linear

- Array / list
- Stack (LIFO)
- Queue (FIFO)
- Linked list

Non-linear and keyed

- Hash table
- Tree (binary search tree, heap)
- Graph
- Set and map

Choosing the right structure affects speed, memory, and code clarity.

Version control and collaboration

Version control tracks changes to code and enables teamwork.

- Git is the most widely used version control system.
- Commit small, focused changes with clear messages.
- Branches isolate work; pull requests support review.
- Platforms: GitHub, GitLab, Bitbucket.

Why it matters

Employers expect familiarity with Git. It protects work, supports collaboration, and documents project history.

Debugging and testing

Debugging

- Reproduce the bug reliably.
- Read error messages carefully.
- Use breakpoints and logging.
- Change one thing at a time.

Testing

- Unit tests for small pieces.
- Integration tests for interactions.
- Manual and automated tests.
- Test behavior and edge cases.

Debugging is a core professional skill. Good tests make change safer.

Ethics and security basics

- Never store passwords in plain text; use hashing and salting.
- Validate input and avoid trusting user data.
- Respect user privacy and data protection laws.
- Write accessible software so more people can use it.
- Avoid copying code without checking its license.
- Report vulnerabilities responsibly.

Mindset

Security, accessibility, and ethics are not optional add-ons. They are part of quality.

Common beginner mistakes

- Trying to learn many languages at once instead of one deeply.
- Copying code without understanding it.
- Ignoring error messages and reading only the last line.
- Skipping version control.
- Not testing edge cases.
- Treating performance as more important than correctness and clarity.
- Avoiding projects; tutorials alone do not build skill.

Practice lab

Task: choose a small problem and take it from idea to working program.

1. Write the algorithm in plain language.
2. Implement it in a high-level language.
3. Test normal and edge cases.
4. Commit your code with Git.
5. Add a short README explaining how to run it.
6. Reflect: what was easy, what was hard, and what you would change.

Success criteria

The program works, the code is readable, tests are documented, and the repository has a clear history.

Recap and next steps

Covered

- Programming basics and career paths
- High-level vs low-level languages
- Compilers, assemblers, linkers, loaders
- Hardware, software, firmware
- RAM vs ROM and memory hierarchy
- CPU components and execution cycle

Suggested next topics

- Data structures and algorithms in depth
- Operating systems and concurrency
- Networking and HTTP
- Databases and SQL
- Version control workflow
- A first real project

Understand the machine. Then build great software.